SAIL-PROXY Developer Guide - Chapter 6

st-gr

2025-01-28

SAIL-PROXY Developer Guide

Multi-provider AI Gateway for SAP AI Core - Developer Documentation **Author:** *st-gr*

<< Previous Chapter | Content Table | Next Chapter >>

Ollama Service

The Ollama service provides full API compatibility with Ollama while routing requests through the main Gateway to SAP AI Core. This enables seamless integration with Ollama-ecosystem tools.

Service Overview (adapted from `/services/ollama/README.md`)

Key Features:

- Full Ollama API compatibility on port 11434
- Localhost-only design for security
- Request transformation to OpenAI format
- Routing through main Gateway service
- Support for chat, generate, embeddings, and model listing

Architecture

Request Flow:

```
Ollama Client → Ollama Service (11434) → Gateway Service (3000) → SAP AI Core
```

Protocol Translation:

```
// Ollama format
{
  "model": "llama2",
  "prompt": "Why is the sky blue?",
  "stream": true
}

// Translated to OpenAI format
{
  "model": "llama2",
  "messages": [{"role": "user", "content": "Why is the sky blue?"}],
  "stream": true
}
```

API Endpoints

Core Ollama Endpoints

Chat API (`/api/chat`):

```
interface OllamaChatRequest {
  model: string;
  messages: Array<{
    role: 'system' | 'user' | 'assistant';
    content: string;
  }>;
  stream?: boolean;
  options?: {
    temperature?: number;
    top_p?: number;
    max_tokens?: number;
  };
}

// Implementation
router.post('/api/chat', async (req, res) => {
  const { model, messages, stream, options } = req.body;

  // Transform to OpenAI format
  const openaiRequest = {
    model,
    messages,
    stream,
    temperature: options?.temperature,
    top_p: options?.top_p,
    max_tokens: options?.max_tokens
  };

  // Forward to Gateway service
  const response = await fetch('http://localhost:3000/openai/v1/chat/completions', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${process.env.INTERNAL_API_KEY}`
    },
    body: JSON.stringify(openaiRequest)
  });

  if (stream) {
    return streamOllamaResponse(res, response);
  } else {
    return transformToOllamaFormat(res, await response.json());
  }
});
```

Generate API (`/api/generate`):

```
interface OllamaGenerateRequest {
  model: string;
  prompt: string;
  stream?: boolean;
  system?: string;
  options?: ModelOptions;
}

router.post('/api/generate', async (req, res) => {
  const { model, prompt, system, stream, options } = req.body;

  // Convert prompt to messages format
```

```
  const messages = [];
  if (system) {
    messages.push({ role: 'system', content: system });
  }
  messages.push({ role: 'user', content: prompt });

  // Transform and forward
  const openaiRequest = {
    model,
    messages,
    stream,
    ...options
  };

  const response = await forwardToGateway(openaiRequest);
  return transformToOllamaGenerate(res, response);
});
```

Model Tags (`/api/tags`):

```
router.get('/api/tags', async (req, res) => {
  try {
    // Get available models from Gateway
    const response = await fetch('http://localhost:3000/v1/models', {
      headers: {
        'Authorization': `Bearer ${process.env.INTERNAL_API_KEY}`
      }
    });

    const { data: models } = await response.json();

    // Transform to Ollama format
    const ollamaModels = models.map(model => ({
      name: model.id,
      modified_at: new Date().toISOString(),
      size: 0, // Size not available from SAP AI Core
      digest: 'sha256:placeholder',
      details: {
        format: 'gguf',
        family: 'llama',
        families: ['llama'],
        parameter_size: '7B',
        quantization_level: 'Q4_0'
      }
    }));

    res.json({ models: ollamaModels });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

OpenAI Compatibility Endpoints

Chat Completions (`/v1/chat/completions`):

```
// Direct pass-through to Gateway with API key injection
router.post('/v1/chat/completions', async (req, res) => {
  const response = await fetch('http://localhost:3000/openai/v1/chat/completions', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${process.env.INTERNAL_API_KEY}`,
      ...req.headers
    },
    body: JSON.stringify(req.body)
```

```
  });

  if (req.body.stream) {
    return streamPassthrough(res, response);
  } else {
    return res.json(await response.json());
  }
});
```

Response Transformation

OpenAI to Ollama Format

Chat Response Transformation:

```
function transformChatResponse(openaiResponse: OpenAIResponse): OllamaResponse {
  const choice = openaiResponse.choices[0];

  return {
    model: openaiResponse.model,
    created_at: new Date().toISOString(),
    message: {
      role: choice.message.role,
      content: choice.message.content
    },
    done: true,
    total_duration: 0,
    load_duration: 0,
    prompt_eval_count: openaiResponse.usage?.prompt_tokens || 0,
    prompt_eval_duration: 0,
    eval_count: openaiResponse.usage?.completion_tokens || 0,
    eval_duration: 0
  };
}
```

Streaming Response Transformation:

```
async function streamOllamaResponse(res: Response, openaiStream: Response): Promise<void> {
  res.writeHead(200, {
    'Content-Type': 'application/x-ndjson',
    'Transfer-Encoding': 'chunked'
  });

  const reader = openaiStream.body?.getReader();
  let buffer = '';

  while (true) {
    const { done, value } = await reader.read();
    if (done) break;

    buffer += new TextDecoder().decode(value);
    const lines = buffer.split('\n');
    buffer = lines.pop() || '';

    for (const line of lines) {
      if (line.startsWith('data: ')) {
        const data = line.slice(6).trim();
        if (data === '[DONE]') {
          res.write(JSON.stringify({ done: true }) + '\n');
          return res.end();
        }

        try {
          const openaiChunk = JSON.parse(data);
          const ollamaChunk = transformStreamChunk(openaiChunk);
```

```
          res.write(JSON.stringify(ollamaChunk) + '\n');
        } catch (error) {
          console.error('Error transforming chunk:', error);
        }
      }
    }
  }
}
```

Development and Testing

Local Development

Start Ollama service:

```
# From project root
pnpm run dev:ollama

# From service directory
cd services/ollama
pnpm run dev

# Service available at http://localhost:11434
```

Testing with Ollama CLI

Install Ollama CLI for testing:

```
# Set Ollama host to point to SAIL-PROXY
export OLLAMA_HOST=http://localhost:11434

# Test model listing
ollama list

# Test chat
ollama run gpt-4o "Hello, how are you?"

# Test streaming
ollama run gpt-4o "Tell me a story" --stream
```

API Testing

Test Ollama endpoints directly:

```
# Test model tags
curl http://localhost:11434/api/tags

# Test chat
curl -X POST http://localhost:11434/api/chat \
  -H "Content-Type: application/json" \
  -d '{
    "model": "gpt-4o",
    "messages": [
      {"role": "user", "content": "Hello!"}
    ]
  }'

# Test generate
curl -X POST http://localhost:11434/api/generate \
  -H "Content-Type: application/json" \
  -d '{
    "model": "gpt-4o",
```

```
    "prompt": "Why is the sky blue?"
  }'
```

Test OpenAI compatibility:

```
# Test OpenAI endpoint
curl -X POST http://localhost:11434/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "gpt-4o",
    "messages": [{"role": "user", "content": "Hello!"}]
  }'
```

Integration with Tools

Continue.dev Integration

Configuration for Continue.dev:

```
{
  "models": [
    {
      "title": "GPT-4o via SAIL-PROXY",
      "provider": "ollama",
      "model": "gpt-4o",
      "apiBase": "http://localhost:11434"
    }
  ]
}
```

Aider Integration

Using Aider with SAIL-PROXY:

```
# Set environment variable
export OLLAMA_API_BASE=http://localhost:11434

# Run aider
aider --model gpt-4o
```

Performance Considerations

Request Overhead

Double-hop latency:

```
Client → Ollama Service → Gateway Service → SAP AI Core
       (localhost)      (localhost)      (network)
```

Optimization strategies:

- Keep Ollama service lightweight
- Minimize transformation overhead
- Use connection pooling for Gateway requests
- Enable keep-alive connections

Memory Management

Stream handling:

```
// Avoid buffering entire responses
const streamWithBackpressure = (source: NodeJS.ReadableStream, target: Response) => {
  source.on('data', (chunk) => {
    if (!target.write(chunk)) {
      source.pause();
      target.once('drain', () => source.resume());
    }
  });
};
```

Security Model

Localhost-Only Design

Security rationale (from project docs):

- Ollama service binds only to localhost (127.0.0.1)
- No external network access allowed
- Relies on Gateway service for authentication
- Internal API key for Gateway communication

API Key Management

Internal authentication:

```
// Use dedicated internal API key
const INTERNAL_API_KEY = process.env.OLLAMA_INTERNAL_API_KEY ||
                         process.env.DEFAULT_API_KEY;

if (!INTERNAL_API_KEY) {
  throw new Error('No internal API key configured for Ollama service');
}
```

Next: Learn about Docker & Deployment strategies and best practices.